MapReduce Laboratory Worksheet

Mini Project: Smart Campus Analytics using Python Hadoop Streaming

This laboratory exercise requires students to design and implement MapReduce jobs for analysing campus activity data using Python. The emphasis is on mapper/reducer design, key-value pairs, composite keys, aggregation, multiple jobs, and interpretation of results.

1. Learning Objectives

- Understand the MapReduce programming model and data flow.
- Design appropriate key-value pairs for a real-world analytics problem.
- Implement Mapper and Reducer components in Python.
- Use composite keys to perform grouped/time-based analysis.
- Chain multiple MapReduce jobs for a non-trivial analytics task.
- Interpret MapReduce output rather than treating the task as only a programming exercise.

2. Problem Statement

A university wants to analyse campus activity using Wi-Fi logs, canteen transactions, library visits, and campus transport records. Develop a Python-based MapReduce application called SmartCampusAnalytics that produces useful operational insights.

3. Input Datasets

Dataset	Fields	Example	Purpose
wifi_logs.csv	StudentID, Timestamp, AccessPoint, DataMB, DurationMin	S101, 2026-08-10 09:15, AP03, 120, 35	Wi-Fi usage
canteen.csv	TransactionID, Timestamp, StudentID, Item, Quantity, Amount	T1001, 2026-08-10 13:05, S101, Sandwich, 2, 160	Purchasing patterns
library.csv	StudentID, Timestamp, EntryType, DurationMin	S101, 2026-08-10 10:20, ENTRY, 85	Library usage
transport.csv	VehicleID, Timestamp, RouteID, SpeedKmph	BUS07, 2026-08-10 08:30, R04, 18.5	Transport congestion

4. Required MapReduce Jobs

Job 1 – Peak Wi-Fi Usage

For each hour, compute total data consumed and total session duration. Identify the peak hour by total DataMB.

Implementation hint: Mapper key: YYYY-MM-DD-HH; value: DataMB, DurationMin, 1. Reducer: sum data, duration, and sessions.

Job 2 – Access Point Utilisation

For each access point, compute total data, total duration, number of sessions, and average session duration.

Implementation hint: Mapper key: AccessPoint; reducer maintains aggregate totals.

Job 3 – Canteen Item Frequency

Find the total quantity sold and total revenue for every item.

Implementation hint: Mapper key: Item; reducer sums Quantity and Amount.

Job 4 – Top Co-Purchased Items

Find pairs of different items appearing in the same transaction and count how many transactions contain each pair.

Implementation hint: Use one MapReduce stage to create sorted item pairs and another to aggregate pair counts.

Job 5 – Library Usage Anomaly

Compute average library duration. Identify students whose average visit duration is more than 1.5 times the overall average.

Implementation hint: Use multiple jobs: first compute per-student totals/counts, then compute overall average and filter students.

Job 6 – Transport Congestion

For every route and hour, calculate average speed. Mark a route-hour as congested when average speed is below 20 km/h.

Implementation hint: Composite mapper key: RouteID-Hour; reducer calculates average speed.

5. Suggested Project Structure

- data/wifi_logs.csv
- data/canteen.csv
- data/library.csv

- data/transport.csv
- src/job1_wifi_mapper.py
- src/job1_wifi_reducer.py
- src/job2_ap_mapper.py
- src/job2_ap_reducer.py
- src/job3_canteen_mapper.py
- src/job3_canteen_reducer.py
- src/job4_stage1_mapper.py
- src/job4_stage1_reducer.py
- src/job4_stage2_mapper.py
- src/job4_stage2_reducer.py
- src/job5_stage1_mapper.py
- src/job5_stage1_reducer.py
- src/job5_global_mapper.py
- src/job5_global_reducer.py
- src/job5_filter_mapper.py
- src/job5_filter_reducer.py
- src/job6_transport_mapper.py
- src/job6_transport_reducer.py

6. Full Reference Implementation – Job 1

The instructor provides Job 1 as the reference implementation. Students should first understand it and then implement the remaining jobs.

6.1 WifiPeakMapper.py

```
import sys
import csv

for line in sys.stdin:
    row = next(csv.reader([line]))
    if not row or row[0] == "StudentID" or len(row) < 5:
        continue
    try:
        hour = row[1].strip()[:13].replace(" ", "-")
        data_mb = float(row[3])
        duration = float(row[4])
        print(f"{hour}\t{data_mb},{duration},1")
    except ValueError:
        continue
```

6.2 WifiPeakReducer.py

```
import sys

for line in sys.stdin:
    key, values = line.rstrip("\n").split("\t", 1)
    total_data = total_duration = 0.0
    sessions = 0
    for v in values.split(";"):
        d, t, n = v.split(",")
        total_data += float(d)
        total_duration += float(t)
        sessions += int(n)

print(f"{key}\tTotalDataMB={total_data:.2f},TotalDurationMin={total_duration:.2f},Sessions={sessions}")
```

7. How to Test the Reference Job Locally

1. Open a terminal in the SmartCampus_MapReduce_Python directory.
2. Verify Python 3: python3 --version
3. Run the mapper independently: cat data/wifi_logs.csv | python3 src/job1_wifi_mapper.py
4. Inspect the mapper key-value pairs.
5. Simulate shuffle and sort: cat data/wifi_logs.csv | python3 src/job1_wifi_mapper.py | sort
6. Run the reducer: cat data/wifi_logs.csv | python3 src/job1_wifi_mapper.py | sort | python3 src/job1_wifi_reducer.py
7. Identify the hour having the maximum TotalDataMB.
8. Record the result in the Expected Output Worksheet.

8. Student Implementation Worksheet

Q1. For Job 1, what should the Mapper output key be? Why?

Answer: _____

_____

Q2. What is the Mapper output value? Can you design a more efficient representation?

Answer: _____

_____

Q3. Why is the hour extracted from the timestamp instead of using the complete timestamp as the key?

Answer: _____

_____

Q4. Which operations can be performed using a Combiner?

Answer: _____

_____

Q5. For Job 2, define the key and value emitted by the Mapper.

Answer: _____

_____

Q6. For Job 4, explain how you will generate every unique pair of products within a transaction.

Answer: _____

_____

Q7. Why must the product pair be sorted, e.g., (Coffee, Sandwich) rather than sometimes (Sandwich, Coffee)?

Answer: _____

_____

Q8. For Job 5, how will you compute the overall average using more than one MapReduce job?

Answer: _____

_____

Q9. For Job 6, explain why RouteID and Hour form a composite key.

Answer: _____

_____

Q10. Which jobs require more than one MapReduce stage? Justify your answer.

Answer: _____

_____

Q11. What happens if a reducer receives millions of values for one key?

Answer: _____

_____

Q12. Explain how data skew could affect the Smart Campus application.

Answer: _____

_____

Q13. Design a custom partitioning strategy for Job 6.

Answer: _____

_____

Q14. What changes are necessary if the input contains malformed records?

Answer: _____

_____

Q15. Suggest one additional campus analytics problem that can be solved using MapReduce.

Answer: _____

_____

9. Expected Output Worksheet

Job	Expected Key	Expected Value	Student Result
Job 1 – Peak Wi-Fi			
Job 2 – Access Point			
Job 3 – Canteen			
Job 4 – Co-Purchase			
Job 5 – Library			
Job 6 – Transport			

10. Viva Questions

- What are the three major phases of MapReduce?
- What is the difference between Mapper output and Reducer output?
- Why does Hadoop sort Mapper output before it reaches the Reducer?
- What is a Combiner and when is it safe to use one?
- What is a Partitioner?
- Can a MapReduce job have zero reducers?
- What happens when there are multiple reducers?
- What is a composite key?
- How would you handle duplicate records?

- Why are multiple MapReduce jobs sometimes necessary?
- What is data locality?
- What is the effect of an excessively large key group?
- How can you reduce network traffic during shuffle?
- How would you test your Mapper independently?
- How would you modify the solution to work with millions of records?

11. Assessment Rubric

Component	Marks	Evaluation
Problem understanding	10	Correct interpretation and assumptions
Mapper design	15	Correct key-value design
Reducer design	15	Correct aggregation/filtering
Multiple-job design	15	Correct job chaining where required
Execution	10	Successful execution
Output correctness	15	Correct results and validation
Efficiency	10	Combiner/partitioning/data-skew awareness
Viva	10	Conceptual understanding

12. Extension Challenges

- Implement a Combiner for the Wi-Fi aggregation job.
- Modify Job 1 to report the top three hours rather than only all hourly totals.
- Implement a custom Partitioner for transport data.
- Rewrite one job using structured serialization instead of comma-separated values.
- Chain Job 3 and Job 4 to identify high-revenue item pairs.
- Introduce malformed records and report the number of rejected records.
- Compare execution with and without a Combiner and explain the observed difference.
- Run the application with multiple reducers and verify that the final output remains correct.

13. Submission Requirements

- Source code for all implemented MapReduce jobs.
- Input datasets used for testing.
- Screenshots of execution and output.
- Completed worksheet.
- A short result interpretation for every job.
- One paragraph discussing performance considerations.
- Viva participation.

14. Instructor Note

The complete Job 1 implementation is provided as a scaffold. For assessment, students should implement Jobs 2–6 themselves. A good laboratory progression is Job 1 → Job 2 → Job 3 → Job 4 → Job 5/6, because this gradually moves from simple aggregation to composite keys and multi-stage MapReduce.